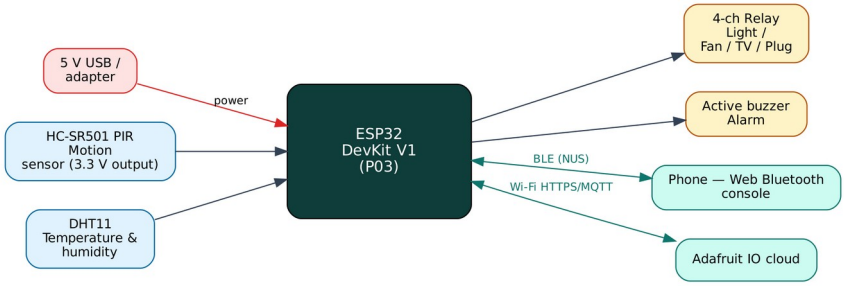
A PROJECT REPORT ON

Smart Home Automation with Away-Mode Security

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (Adafruit IO)



Project: P03 · Intermediate level

Peripherals: 4-channel relay module, HC-SR501 PIR sensor, DHT11, Active buzzer

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **“Smart Home Automation with Away-Mode Security”** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract 6

1. Introduction 7

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System 8

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design 9

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design 11

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams 13

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 15

6. Software Design and Implementation 16

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 17
6.6 Cloud Integration (Adafruit IO) 18
6.7 Configurable Settings 18

7. Setup and Deployment Guide 19

7.1 Hardware Assembly 19
7.2 Flashing the Firmware 19
7.3 Cloud Setup (Adafruit IO) 19
7.4 BLE App Setup (Web Bluetooth Console) 20
7.5 First-Run Checklist 21

8. Testing and Results 22

8.1 Expected Output 22
8.2 Troubleshooting 22

9. Discussion 23

9.1 Advantages 23
9.2 Limitations 23
9.3 Safety Precautions 23
9.4 Security Considerations 23
9.5 Future Scope 23

10. Conclusion **24**

References **25**

Appendix A: Project Source Code **26**

Appendix B: Shared Library Code **30**

Abstract

This project converts four ordinary electrical appliances into smart appliances that can be controlled from a phone over Bluetooth Low Energy or from anywhere in the world through the Adafruit IO cloud dashboard. A 4-channel relay module switches a light, a fan, a TV socket and a spare plug. A PIR motion sensor and a DHT11 temperature-humidity sensor enable automation: in AUTO mode the light turns on when someone enters and off after a period without motion, and the fan follows the room temperature with hysteresis. In AWAY mode the same motion sensor becomes an intruder detector that sounds a buzzer and publishes an alert. Relay states are saved in flash and restored after power cuts, and the cloud and BLE views are kept in sync.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, Adafruit IO, 4-channel relay module, HC-SR501 PIR sensor, DHT11, Active buzzer.

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through Adafruit IO, a free IoT/cloud platform.

1.1 Problem Statement

Commercial smart switches need Wi-Fi and a vendor cloud; when the internet is down they cannot be operated from the phone at all. Lights and fans are often left on in empty rooms, wasting energy, and homes left empty during travel have no low-cost intrusion warning.

1.2 Objectives

1. Control four appliances from a phone over BLE (local) and over the internet (Adafruit IO).
2. Automate the light by occupancy and the fan by temperature.
3. Provide an AWAY security mode with buzzer and cloud alert on motion.
4. Restore appliance states after a power failure.
5. Keep local, cloud and physical states consistent.

1.3 Scope of the Project

A single room or small flat with up to four loads of up to 10 A each. Voice assistants, energy metering and multi-room networking are out of scope but can be added through Adafruit IO integrations.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Bluetooth-only relay boards work only within range, while Wi-Fi smart plugs depend on the internet and on the vendor's servers. Few low-cost systems combine automation rules, security and remote control in one device.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Hybrid BLE + Wi-Fi control: either path works if the other is down.
- Three modes (MANUAL, AUTO, AWAY) selectable from the phone or the dashboard.
- Occupancy-based lighting with a configurable timeout and temperature-based fan control with 1 °C hysteresis.
- Persistent state in NVS flash and cloud state retrieval after reconnect.

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

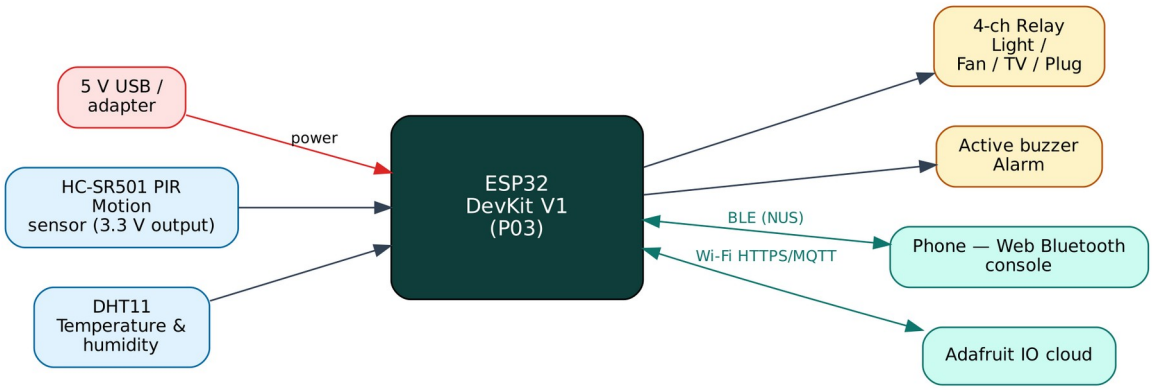


Figure 1: Block diagram of the Smart Home Automation

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the Adafruit IO cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — 4-channel relay module, HC-SR501 PIR sensor, DHT11, Active buzzer connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted MQTT to Adafruit IO.
- **Application layer** — the Web Bluetooth console on the phone and the Adafruit IO dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	4-channel 5 V relay module (10 A)	1	Opto-isolated, active-LOW	₹170
5	HC-SR501 PIR motion sensor	1	3-7 m range, adjustable delay	₹70
6	DHT11 temperature & humidity module	1	0-50 °C, 20-90 % RH	₹60
7	Active buzzer 5 V	1	Built-in oscillator, ~85 dB	₹15
8	5 V 2 A adapter	1	Relays draw up to ~300 mA	₹150
	Total (approx.)			**₹1025**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹465.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

Relay module

Each channel's opto-coupler isolates the ESP32 from the relay coil circuit. A LOW signal turns the relay ON, and its changeover contacts switch the appliance's live wire.

HC-SR501 PIR

A pyroelectric sensor behind a Fresnel lens detects changes in infrared radiation when a warm body moves across its zones. The BISS0001 chip outputs a 3.3 V HIGH pulse while motion continues.

DHT11

Measures temperature with an NTC and humidity with a capacitive element, sending a 40-bit digital frame.

Active buzzer

Contains its own oscillator, so a DC HIGH level produces a tone; the firmware pulses it at 1 Hz for an alarm pattern.

4. Circuit Design

4.1 Circuit Diagram

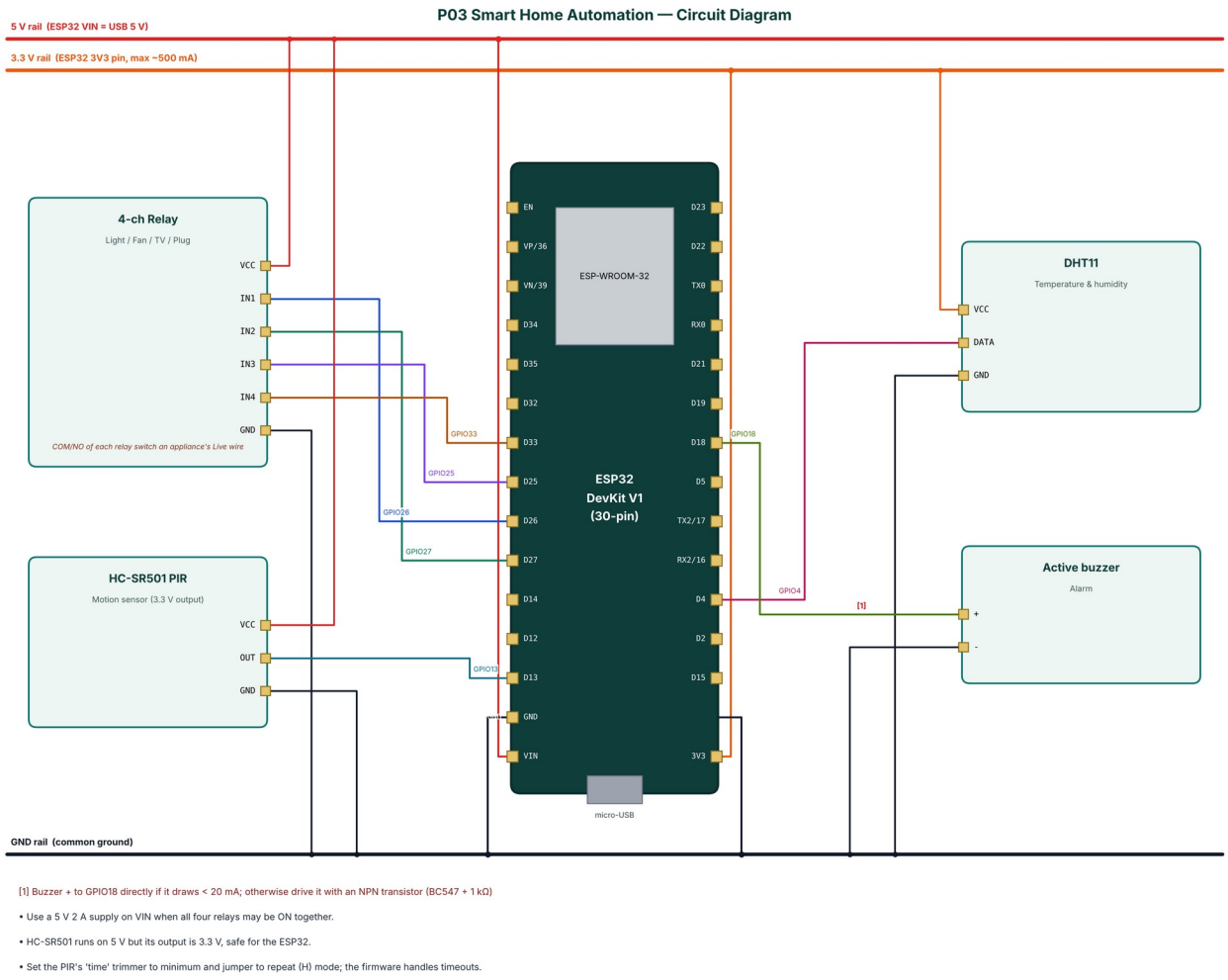


Figure 2: Circuit diagram of the Smart Home Automation

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
4-ch Relay	VCC	5V rail (VIN)	
4-ch Relay	IN1	GPIO26	
4-ch Relay	IN2	GPIO27	
4-ch Relay	IN3	GPIO25	
4-ch Relay	IN4	GPIO33	
4-ch Relay	GND	GND	
HC-SR501 PIR	VCC	5V rail (VIN)	
HC-SR501 PIR	OUT	GPIO13	
HC-SR501 PIR	GND	GND	
DHT11	VCC	3.3 V (3V3 pin)	
DHT11	DATA	GPIO4	

Module	Module pin	ESP32 connection	Notes
DHT11	GND	GND	
Active buzzer	+	GPIO18	Buzzer + to GPIO18 directly if it draws < 20 mA; otherwise drive it with an NPN transistor (BC547 + 1 kΩ)
Active buzzer	-	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- Use a 5 V 2 A supply on VIN when all four relays may be ON together.
- HC-SR501 runs on 5 V but its output is 3.3 V, safe for the ESP32.
- Set the PIR's 'time' trimmer to minimum and jumper to repeat (H) mode; the firmware handles timeouts.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

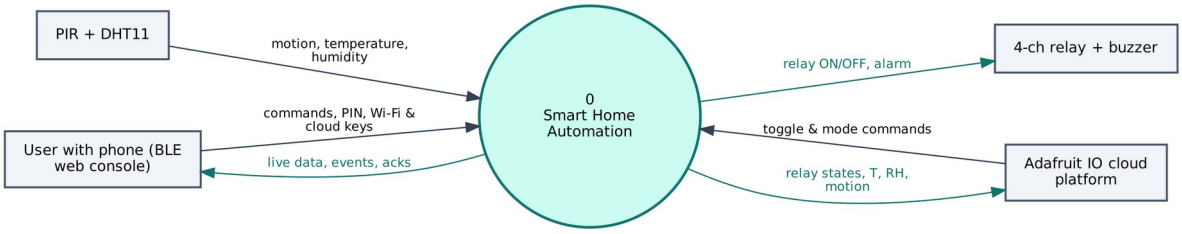


Figure 3: DFD Level 0 — context diagram

At level 0 the whole Smart Home Automation is a single process. It receives motion, temperature, humidity from the PIR + DHT11, commands and configuration from the user's phone, and commands from Adafruit IO. It sends relay ON/OFF, alarm to the 4-ch relay + buzzer, live data to the phone and relay states, T, RH, motion to Adafruit IO.

5.2 DFD Level 1

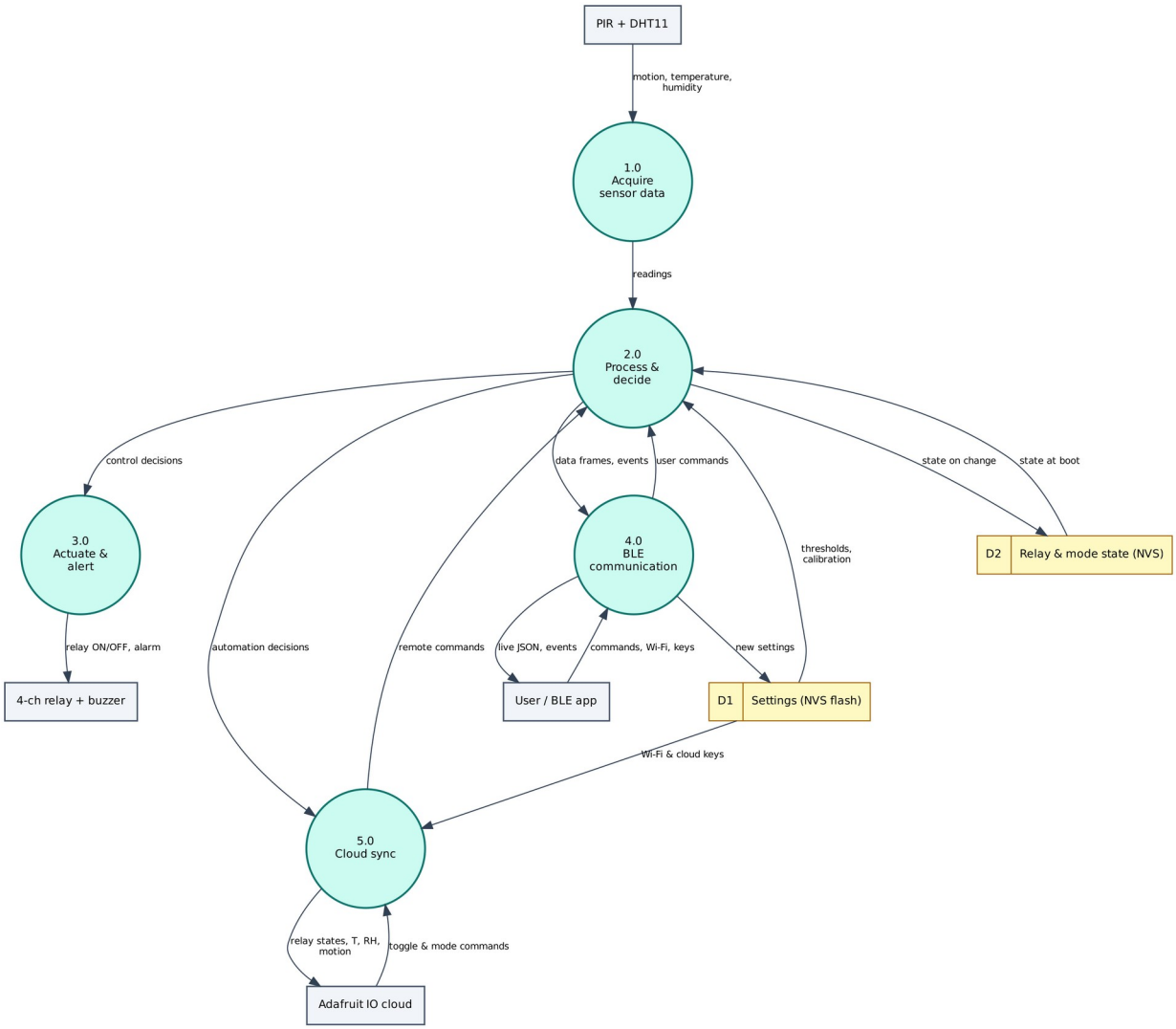


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads motion, temperature, humidity from the PIR + DHT11; filters/averages the raw values.
2.0 Process & decide	Computes automation decisions; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the 4-ch relay + buzzer (relay ON/OFF, alarm).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends relay states, T, RH, motion to Adafruit IO; receives toggle & mode commands.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Relay & mode state (NVS)	Stores state on change; returns state at boot.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
t	Temperature	℃
h	Humidity	%
m	Motion now	bool
mode	MANUAL / AUTO / AWAY	—
alarm	Alarm sounding	bool
r	Relay states [light, fan, tv, plug]	0/1
cloud	MQTT connected	bool

Table 5: BLE data frame fields

Cloud object	Content / usage
p03-light / p03-fan / p03-tv / p03-plug	Toggle blocks with ON/OFF values
p03-mode	Text or multi-button block: AUTO, MANUAL, AWAY
p03-temperature, p03-humidity	Gauge / line chart
p03-motion	Indicator or stream block (MOTION / CLEAR / INTRUSION)

Table 6: Adafruit IO data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	DHT sensor library (Adafruit), Adafruit Unified Sensor
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	Adafruit IO (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P03_Smart_Home_Automation.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

At start-up the firmware restores the last relay states and mode from NVS flash, starts BLE and connects to Adafruit IO over MQTT/TLS (port 8883). It subscribes to five feeds — p03-light, p03-fan, p03-tv, p03-plug and p03-mode — and asks the broker for each feed's last value, so changes made on the dashboard while the device was offline are applied.

In AUTO mode, motion turns the light on immediately; it turns off after 'lightto' seconds (default 60) without motion. The fan turns on when the temperature reaches 'fanon' (default 30 °C) and off below fanon − 1 °C. In AWAY mode motion starts a 60-second buzzer pattern, publishes INTRUSION on p03-motion and sends an 'intrusion' event to the phone. MANUAL mode disables all rules.

Every relay change, from any source, is written to flash and published to its feed. Messages that echo back from the broker are ignored because the state already matches. Temperature and humidity are published every 30 s.

6.4 Flowchart

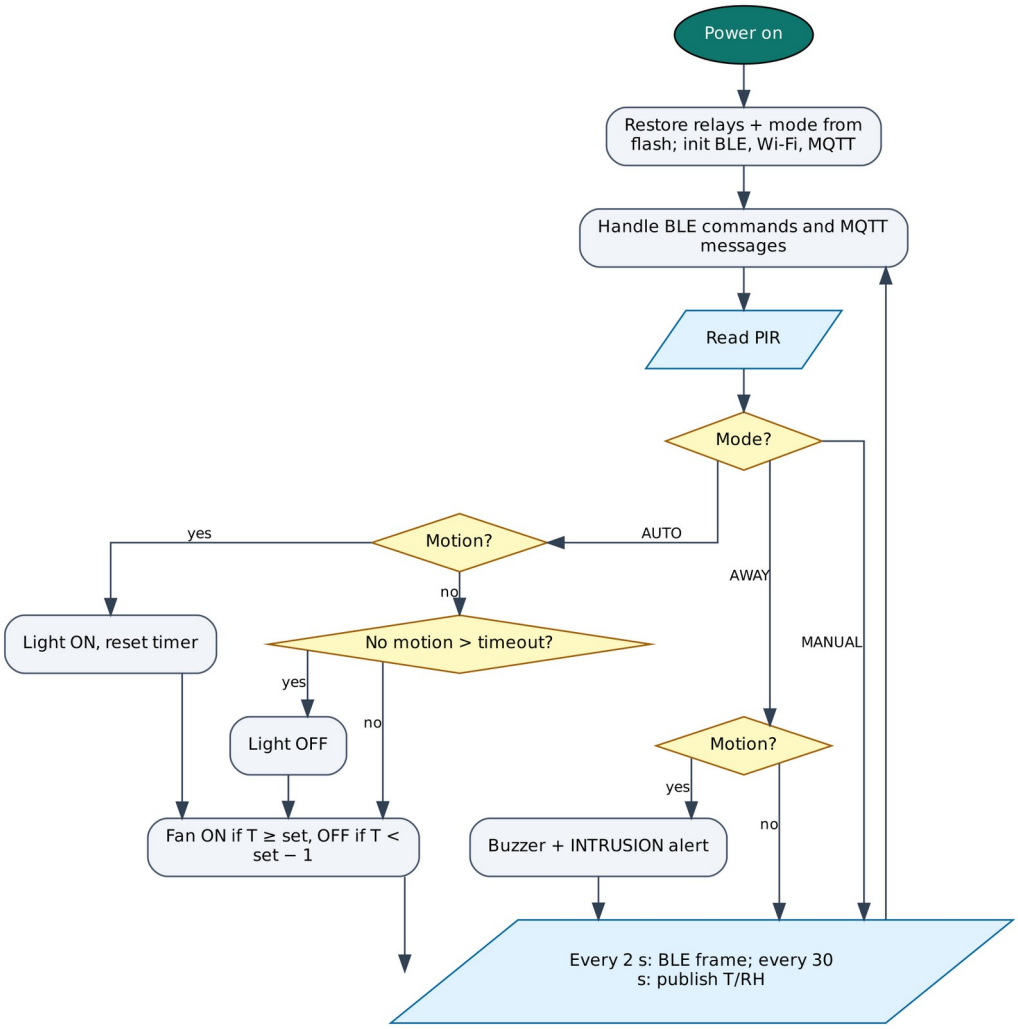


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P03-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
r1:on|off|toggle … r4:…	Yes	Switch one appliance
all:off / all:on	Yes	Switch all four
mode:auto|manual|away	Yes	Change mode
alarm:off	Yes	Silence the alarm

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"t":30.1,"h":58,"m":true,"mode":"AUTO","alarm":false,"r":[1,1,0,0],"cloud":true,"type":"data"}
```

6.6 Cloud Integration (Adafruit IO)

The device keeps a persistent MQTT connection to io.adafruit.com on port 8883 (TLS) using the account username and key. Values are published to `username/feeds/<feed-key>`, and control feeds are subscribed so dashboard changes arrive within about a second. A built-in limiter keeps the device below Adafruit IO's free limit of 30 data points per minute.

Object	Content
p03-light / p03-fan / p03-tv / p03-plug	Toggle blocks with ON/OFF values
p03-mode	Text or multi-button block: AUTO, MANUAL, AWAY
p03-temperature, p03-humidity	Gauge / line chart
p03-motion	Indicator or stream block (MOTION / CLEAR / INTRUSION)

Table 10: Adafruit IO configuration

6.7 Configurable Settings

Key	Default	Meaning
aiouser / aiokey	—	Adafruit IO credentials
lightto	60	Light-off timeout (s)
fanon	30	Fan-on temperature (°C)
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **DHT sensor library (Adafruit)**, **Adafruit Unified Sensor**.
5. Download the repository and open the folder **P03_Smart_Home_Automation** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P03_Smart_Home_Automation** and unzip it to get `P03_Smart_Home_Automation.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (Adafruit IO)

1. Go to **io.adafruit.com** and create a free account.
2. Click the yellow **key icon (My Key / Adafruit IO Key)** and copy your **Username** and **Active Key**.

3. In the BLE web console open **Setup**, unlock and save them in `aiouser` and `aiokey` (commands: `set:aiouser=NAME`, `set:aiokey=aio_xxxx`).
4. Power the device: feeds are created automatically on the first publish. You can also create them first in **Feeds → New Feed** using the exact keys in the table below.
5. Open **Dashboards → New Dashboard**, then **+ (Create New Block)**. Use Toggle blocks for switch feeds with button-on text `ON` and button-off text `OFF` (or `AUTO`/`MANUAL` for mode feeds), Gauge blocks for levels and Line Chart blocks for history.
6. Free tier: 30 data points per minute, 10 feeds, 5 dashboards and 30 days of history per account. Projects on the same account share these limits, so delete feeds of projects you are not running or use separate accounts.

Field / feed / table	Content
p03-light / p03-fan / p03-tv / p03-plug	Toggle blocks with ON/OFF values
p03-mode	Text or multi-button block: AUTO, MANUAL, AWAY
p03-temperature, p03-humidity	Gauge / line chart
p03-motion	Indicator or stream block (MOTION / CLEAR / INTRUSION)

Table 12: Items to create in Adafruit IO

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P03** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P03-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the Adafruit IO keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P03-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P03-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in Adafruit IO within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	BLE switching	Unlock, then toggle each switch in the web console	Correct relay clicks; LED on relay board lights	
T2	Cloud switching	Toggle p03-fan on the Adafruit IO dashboard	Relay 2 follows within ~1 s	
T3	Occupancy light	Set lightto=20, walk past the PIR and leave	Light ON at once, OFF ~20 s after the last motion	
T4	Fan automation	Set fanon to 1 °C below room temperature	Fan turns ON; raise fanon by 2 °C and it turns OFF	
T5	Away alarm	Select AWAY and move in front of the PIR	Buzzer beeps; 'intrusion' event and INTRUSION on p03-motion	
T6	Power cut	Turn relays 1 and 3 ON, unplug and re-plug	Relays 1 and 3 come back ON	
T7	Internet down	Disconnect router, control over BLE	BLE control keeps working; cloud resyncs when back	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"t":30.1,"h":58,"m":true,"mode":"AUTO","alarm":false,"r":[1,1,0,0],"cloud":true,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in Adafruit IO	Wrong username/key, rate limit exceeded or 10-feed limit reached.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- Works over BLE when the internet is down and over the internet when away.
- Saves energy with occupancy and temperature rules.
- Adds basic security at no extra cost by reusing the PIR.
- State survives power cuts.
- Uses a free cloud with a customisable dashboard.

9.2 Limitations

- The PIR cannot detect a person sitting completely still, so the light may switch off.
- Adafruit IO free tier allows 10 feeds in total per account (this project uses 8).
- Relays are not suitable for dimming or fan-speed control.
- No physical wall-switch input in this version.

9.3 Safety Precautions

- Appliance wiring is at 230 V AC: use proper screw terminals, insulation and an enclosure.
- Do not exceed the relay rating (10 A resistive; derate for motors).
- Keep the low-voltage ESP32 wiring physically separated from mains wiring.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Add wall-switch inputs so manual switches and the app stay in sync.
- Add voice control through Adafruit IO's IFTTT / Google Assistant integrations.
- Use a triac or TRIAC-based dimmer for light dimming and fan speed control.
- Add a camera (ESP32-CAM) snapshot on intrusion.

10. Conclusion

The system delivers practical smart-home features — remote and local control, occupancy lighting, temperature-based ventilation and an intrusion alarm — with a ₹1,000 bill of materials. Combining BLE and cloud control removes the single point of failure of internet-only smart switches, and state persistence makes it dependable in areas with frequent power cuts.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. Murata/BISS0001 PIR controller and HC-SR501 module datasheet.
2. Aosong, DHT11 datasheet.
3. Adafruit Industries, Adafruit IO MQTT API — '/get' topic and rate limits.
4. Songle SRD-05VDC-SL-C relay datasheet.
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. Adafruit Industries, Adafruit IO documentation and MQTT API, io.adafruit.com/api/docs.

Appendix A: Project Source Code

The complete code is in the repository folder **P03_Smart_Home_Automation** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P03 Smart Home Automation — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define AIO_USERNAME  ""
#define AIO_KEY       ""

// ----- pins -----
#define PIN_RELAY1    26     // Light
#define PIN_RELAY2    27     // Fan
#define PIN_RELAY3    25     // TV / Socket
#define PIN_RELAY4    33     // Spare plug
#define RELAY_ACTIVE_LOW true   // most blue 4-channel modules switch ON with LOW
#define PIN_PIR       13     // HC-SR501 OUT
#define PIN_DHT       4      // DHT11 data
#define PIN_BUZZER    18     // active buzzer (+) via NPN or directly (< 20 mA)

// ----- automation defaults (change over BLE with set:key=value) -----
#define DEF_LIGHT_TIMEOUT_S  60   // lightto : light off after no motion
#define DEF_FAN_ON_C         30   // fanon   : fan on above this temperature
#define SENSOR_MS            2000
#define CLOUD_MS             30000
```

P03_Smart_Home_Automation.ino

```
/*
 * P03 — Smart Home Automation with Away-Mode Security
 * ===================================================
 * Peripherals: 4-channel relay module, HC-SR501 PIR, DHT11, active buzzer
 * Wi-Fi  : Adafruit IO (MQTT) — 4 toggle feeds control the relays from the
 *          dashboard / phone; temperature, humidity and motion are published
 * BLE    : local control when the internet is down, live status, mode change
 * Modes  : MANUAL - relays only change by command
 *          AUTO   - PIR switches the light, temperature switches the fan
 *          AWAY   - any motion sounds the buzzer and raises an intrusion alert
 * Extras : relay states survive power cuts (saved in flash), fan hysteresis,
 *          cloud <-> BLE state stays in sync
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <DHT.h>

DHT dht(PIN_DHT, DHT11);
cloud::AdafruitIO aio;

const int RELAY_PINS[4] = { PIN_RELAY1, PIN_RELAY2, PIN_RELAY3, PIN_RELAY4 };
const char* RELAY_NAMES[4] = { "Light", "Fan", "TV", "Plug" };
const char* RELAY_FEEDS[4] = { "p03-light", "p03-fan", "p03-tv", "p03-plug" };
const char* SUB_FEEDS[] = { "p03-light", "p03-fan", "p03-tv", "p03-plug", "p03-mode" };

enum Mode { MANUAL, AUTO, AWAY };
const char* MODE_NAMES[] = { "MANUAL", "AUTO", "AWAY" };
```

```
Mode mode = AUTO;

bool relay[4] = { false, false, false, false };
bool motion = false, alarmOn = false;
float temp = NAN, hum = NAN;
uint32_t lastMotion = 0, lastSensor = 0, lastCloud = 0, alarmStart = 0;
uint32_t lightTimeoutMs = DEF_LIGHT_TIMEOUT_S * 1000UL;
float fanOnC = DEF_FAN_ON_C;

void loadSettings() {
  lightTimeoutMs = iot::settingF("lightto", DEF_LIGHT_TIMEOUT_S) * 1000UL;
  fanOnC = iot::settingF("fanon", DEF_FAN_ON_C);
}

void applyRelay(int i) {
  digitalWrite(RELAY_PINS[i], (relay[i] ^ RELAY_ACTIVE_LOW) ? HIGH : LOW);
}

void setRelay(int i, bool on, const char* source, bool publish = true) {
  if (i < 0 || i > 3 || relay[i] == on) return;
  relay[i] = on;
  applyRelay(i);
  iot::saveSetting(("r" + String(i + 1)).c_str(), on ? "1" : "0");
  iot::sendEvent("relay", String(RELAY_NAMES[i]) + (on ? " ON" : " OFF") + " (" + source + ")");
  if (publish) aio.publish(RELAY_FEEDS[i], on ? "ON" : "OFF");
}

void setMode(Mode m, const char* source) {
  mode = m;
  iot::saveSetting("mode", String((int)m));
  if (m != AWAY) { alarmOn = false; digitalWrite(PIN_BUZZER, LOW); }
  iot::sendEvent("mode", String(MODE_NAMES[m]) + " (" + source + ")");
  aio.publish("p03-mode", MODE_NAMES[m]);
}

void automation() {
  uint32_t now = millis();
  if (mode == AUTO) {
    if (motion && !relay[0]) setRelay(0, true, "motion");
    if (!motion && relay[0] && now - lastMotion > lightTimeoutMs) setRelay(0, false, "no motion");
    if (!isnan(temp)) {
      if (temp >= fanOnC && !relay[1]) setRelay(1, true, "temperature");
      if (temp < fanOnC - 1.0 && relay[1]) setRelay(1, false, "temperature");   // 1 °C hysteresis
    }
  }
  if (mode == AWAY && motion && !alarmOn) {
    alarmOn = true; alarmStart = now;
    iot::sendEvent("intrusion", "Motion detected while AWAY");
    aio.publish("p03-motion", "INTRUSION");
  }
  // buzzer pattern: 1 Hz beeping for up to 60 s
  if (alarmOn) {
    digitalWrite(PIN_BUZZER, ((now - alarmStart) / 500) % 2);
    if (now - alarmStart > 60000) { alarmOn = false; digitalWrite(PIN_BUZZER, LOW); }
  }
}

void sendBle() {
  JsonDocument d;
  d["t"] = iot::rnd(temp); d["h"] = iot::rnd(hum, 0);
  d["m"] = motion; d["mode"] = MODE_NAMES[mode]; d["alarm"] = alarmOn;
  JsonArray r = d["r"].to<JsonArray>();
  for (int i = 0; i < 4; i++) r.add(relay[i] ? 1 : 0);
```

```
  d["cloud"] = aio.connected();
  iot::sendData(d);
}

void onFeed(const String& feed, const String& value) {
  bool on = value.equalsIgnoreCase("ON") || value == "1";
  for (int i = 0; i < 4; i++)
    if (feed == RELAY_FEEDS[i]) { setRelay(i, on, "cloud", false); sendBle(); return; }
  if (feed == "p03-mode") {
    for (int m = 0; m < 3; m++)
      if (value.equalsIgnoreCase(MODE_NAMES[m]) && mode != m) { mode = (Mode)m; setMode(mode,
"cloud"); }
  }
}

// Commands: r1:on  r2:off  r3:toggle  all:off  mode:auto|manual|away  alarm:off
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") { loadSettings(); return true; }
  bool isRelay = cmd.length() == 2 && cmd[0] == 'r' && cmd[1] >= '1' && cmd[1] <= '4';
  if (!(isRelay || cmd == "all" || cmd == "mode" || cmd == "alarm")) return false;
  if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
  if (isRelay) {
    int i = cmd[1] - '1';
    bool on = arg == "on" ? true : arg == "off" ? false : !relay[i];
    setRelay(i, on, "BLE");
  } else if (cmd == "all") {
    for (int i = 0; i < 4; i++) setRelay(i, arg == "on", "BLE");
  } else if (cmd == "mode") {
    String a = arg; a.toUpperCase();
    for (int m = 0; m < 3; m++) if (a == MODE_NAMES[m]) setMode((Mode)m, "BLE");
  } else if (cmd == "alarm") {
    alarmOn = false; digitalWrite(PIN_BUZZER, LOW);
  }
  iot::emitAck(cmd, true);
  sendBle();
  return true;
}

void setup() {
  for (int i = 0; i < 4; i++) { pinMode(RELAY_PINS[i], OUTPUT); }
  pinMode(PIN_PIR, INPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  iot::begin("P03", onCommand);
  // restore last relay states and mode after a power cut
  for (int i = 0; i < 4; i++) { relay[i] = iot::setting(("r" + String(i + 1)).c_str(), "0") == "1";
applyRelay(i); }
  mode = (Mode)constrain((int)iot::settingF("mode", AUTO), 0, 2);
  loadSettings();
  dht.begin();
  // On every (re)connect the library asks Adafruit IO for the last value of
  // each subscribed feed, so changes made on the dashboard while the board
  // was offline are applied (the cloud is the source of truth).
  aio.begin(onFeed, SUB_FEEDS, 5);
}

void loop() {
  iot::loop();
  aio.loop();
  uint32_t now = millis();

  bool m = digitalRead(PIN_PIR) == HIGH;
  if (m) lastMotion = now;
  if (m != motion) {
```

```
    motion = m;
    if (mode != AWAY) aio.publish("p03-motion", m ? "MOTION" : "CLEAR");
    sendBle();
  }
  automation();

  if (now - lastSensor >= SENSOR_MS) {
    lastSensor = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (!isnan(t)) temp = t;
    if (!isnan(h)) hum = h;
    sendBle();
  }
  if (aio.connected() && now - lastCloud >= CLOUD_MS && !isnan(temp)) {
    lastCloud = now;
    aio.publish("p03-temperature", temp, 1);
    aio.publish("p03-humidity", hum, 0);
  }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ------------------------------------------------------------------ Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  —  free-tier cloud helpers used by the projects
 * ---------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ================================================================ ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ================================================================ Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                          String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```